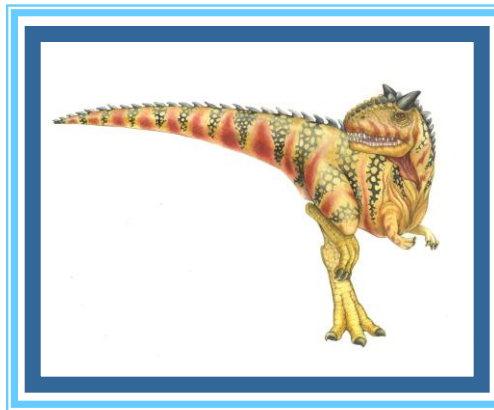


Capítulo 3: Processos





Capítulo 3: Processos

- ❑ Conceito de Processo
- ❑ Agendamento de Processos
- ❑ Operações em Processos
- ❑ Comunicação entre processos
- ❑ Exemplos de sistemas IPC
- ❑ Comunicação em sistemas cliente-servidor





Objetivos

- ❑ Para introduzir a noção de um processo — um programa em execução, que constitui a base de toda a computação
- ❑ Descrever as várias características dos processos, incluindo agendamento, criação e término e comunicação
- ❑ Explorar a comunicação entre processos usando memória compartilhada e passagem de mensagens
- ❑ Para descrever a comunicação em sistemas cliente-servidor





Conceito de Processo

- Um sistema operacional executa uma variedade de programas:
 - Sistema de lote – **empregos**
 - Sistemas de tempo compartilhado – **programas** ou **tarefas do usuário**
- O livro didático usa os termos **trabalho** e **processo** quase indistintamente
- **Processo** – um programa em execução; a execução do processo deve progredir de forma sequencial
- Várias partes
 - O código do programa, também chamado de **seção de texto**
 - Atividade atual incluindo **programa contador** , registradores do processador
 - **Pilha** contendo dados temporários
 - ▶ Parâmetros de função, endereços de retorno, variáveis locais
 - **Seção de dados** contendo variáveis globais
 - **Monte** contendo memória alocada dinamicamente durante o tempo de execução





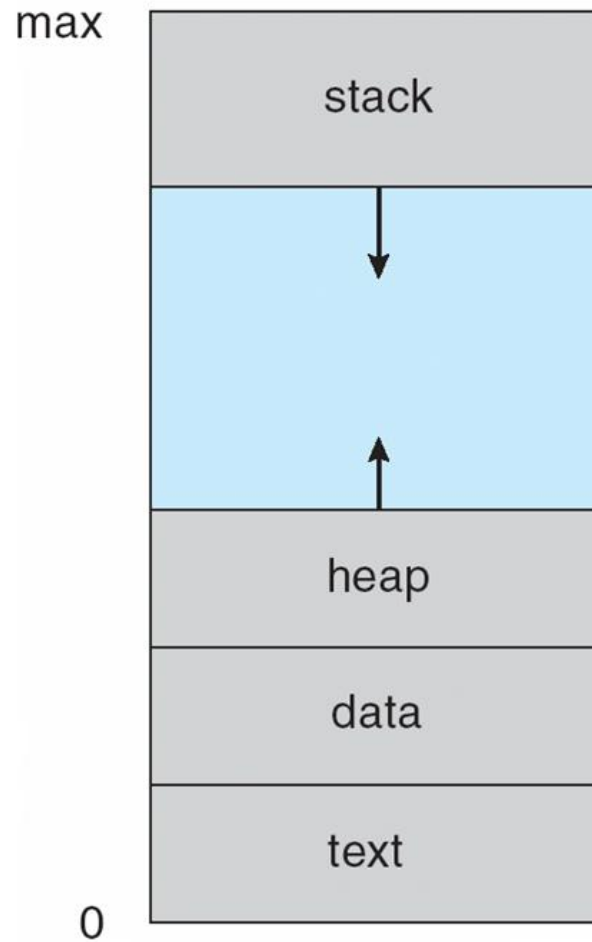
Conceito de Processo (Cont.)

- ❑ O programa é uma entidade **passiva** armazenada no disco (**arquivo executável**), o processo é **ativo**
 - ❑ O programa se torna um processo quando o arquivo executável é carregado na memória
- ❑ Execução do programa iniciada por meio de cliques do mouse na GUI, entrada de linha de comando de seu nome, etc.
- ❑ Um programa pode ser vários processos
 - ❑ Considere vários usuários executando o mesmo programa





Processo na Memória





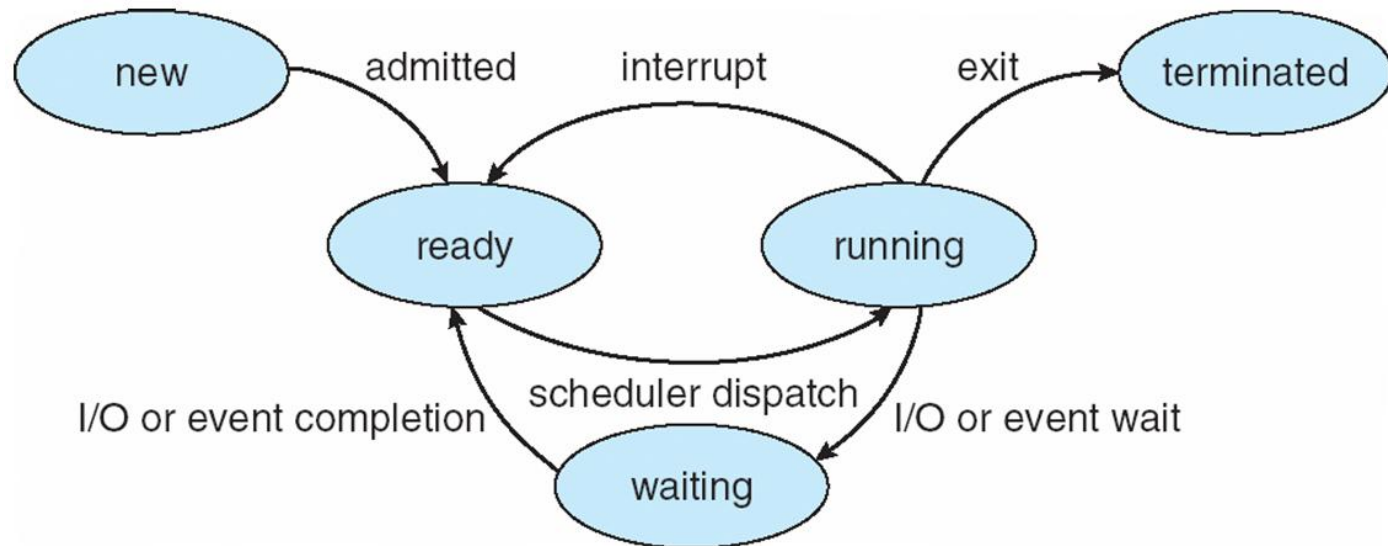
Estado do processo

- À medida que um processo é executado, ele muda **de estado**
 - **novo** : O processo está sendo criado
 - **running** : As instruções estão sendo executadas
 - **esperando** : O processo está esperando que algum evento ocorra
 - **pronto** : O processo está esperando para ser atribuído a um processador
 - **terminado** : O processo concluiu a execução





Diagrama do estado do processo

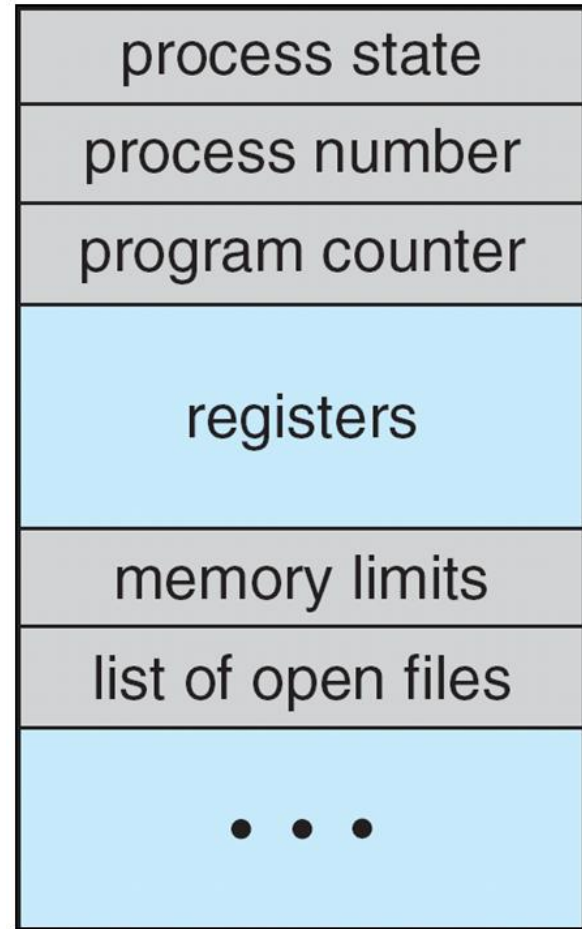




Bloco de controle de processo (PCB)

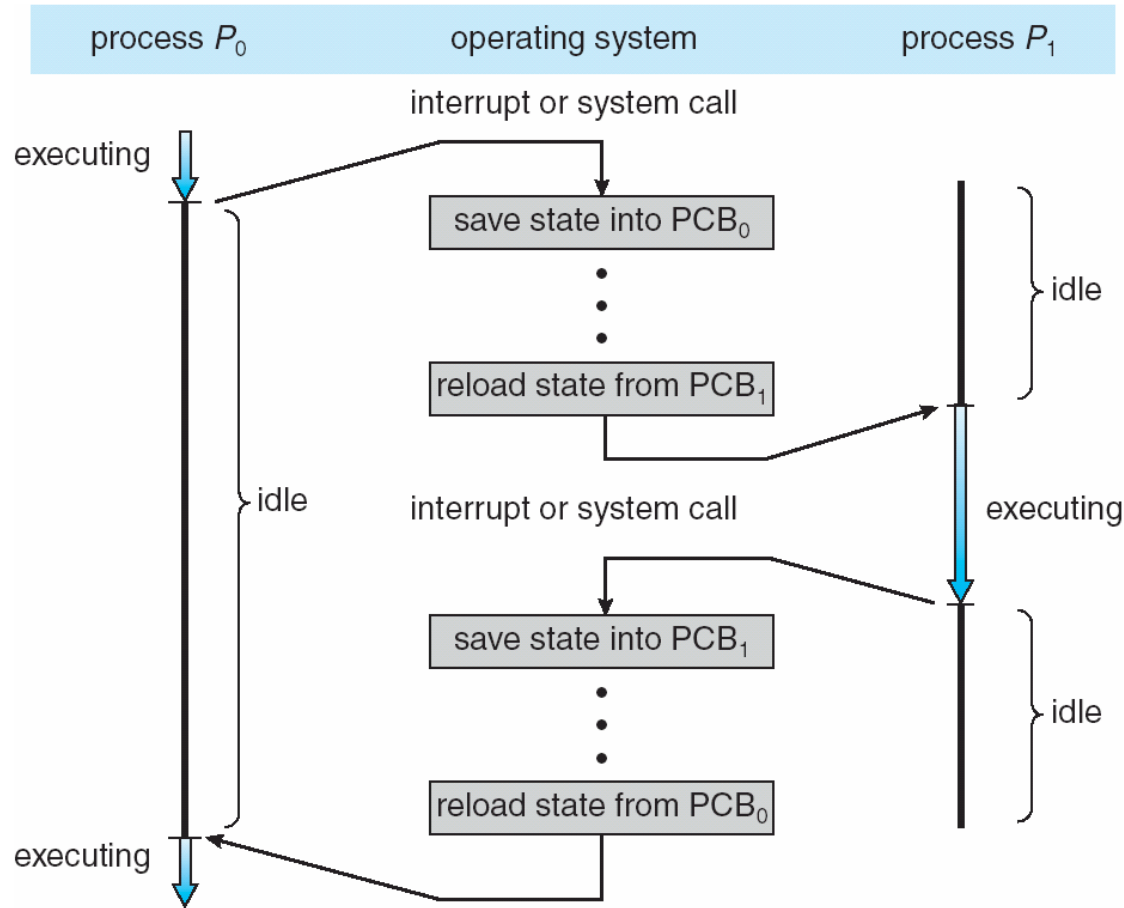
Informações associadas a cada processo
(também chamado de **bloco de controle de tarefas**)

- ❑ Estado do processo – em execução, em espera, etc.
- ❑ Contador de programa – localização da instrução a ser executada em seguida
- ❑ Registradores da CPU – conteúdo de todos os registradores centrados no processo
- ❑ Informações de agendamento da CPU - prioridades, ponteiros de fila de agendamento
- ❑ Informações de gerenciamento de memória – memória alocada ao processo
- ❑ Informações contábeis – CPU usada, tempo decorrido desde o início, limites de tempo
- ❑ Informações de status de E/S – dispositivos de E/S alocados para processar, lista de arquivos abertos





Troca de CPU de processo para processo





Tópicos

- ❑ Até agora, o processo tem um único thread de execução
- ❑ Considere ter vários contadores de programa por processo
 - ❑ Vários locais podem ser executados ao mesmo tempo
 - ▶ Vários threads de controle -> **threads**
- ❑ Deve então ter armazenamento para detalhes de thread, vários contadores de programa em PCB
- ❑ Veja o próximo capítulo

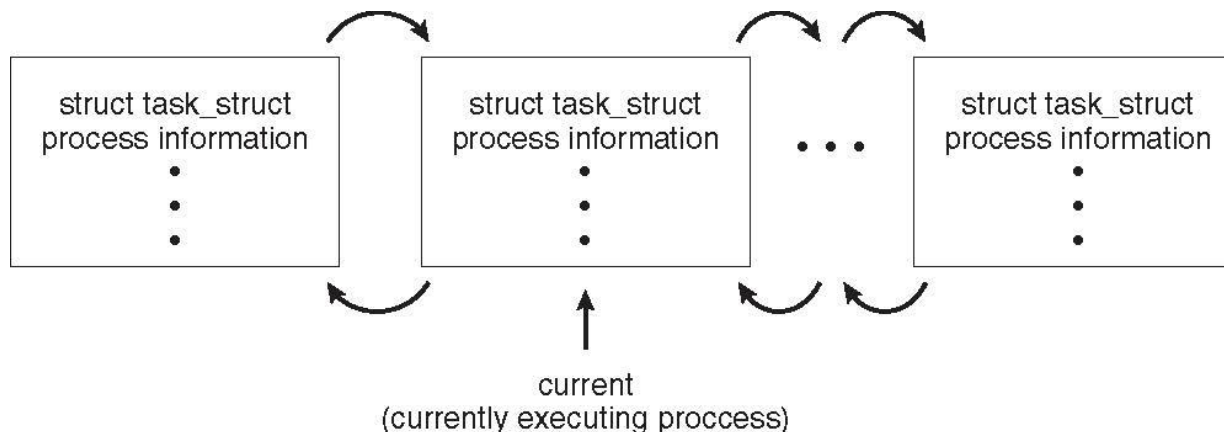




Representação de Processos no Linux

Representado pela estrutura C `task_struct`

```
pid t_pid; /* identificador do processo */
long state; /* estado do processo */
unsigned int time_slice /* informações de agendamento */
struct task_struct *parent; /* pai deste processo */
struct list_head children; /* filhos deste processo */
struct files_struct *files; /* lista de arquivos abertos */
struct mm_struct *mm; /* espaço de endereço deste processo */
```





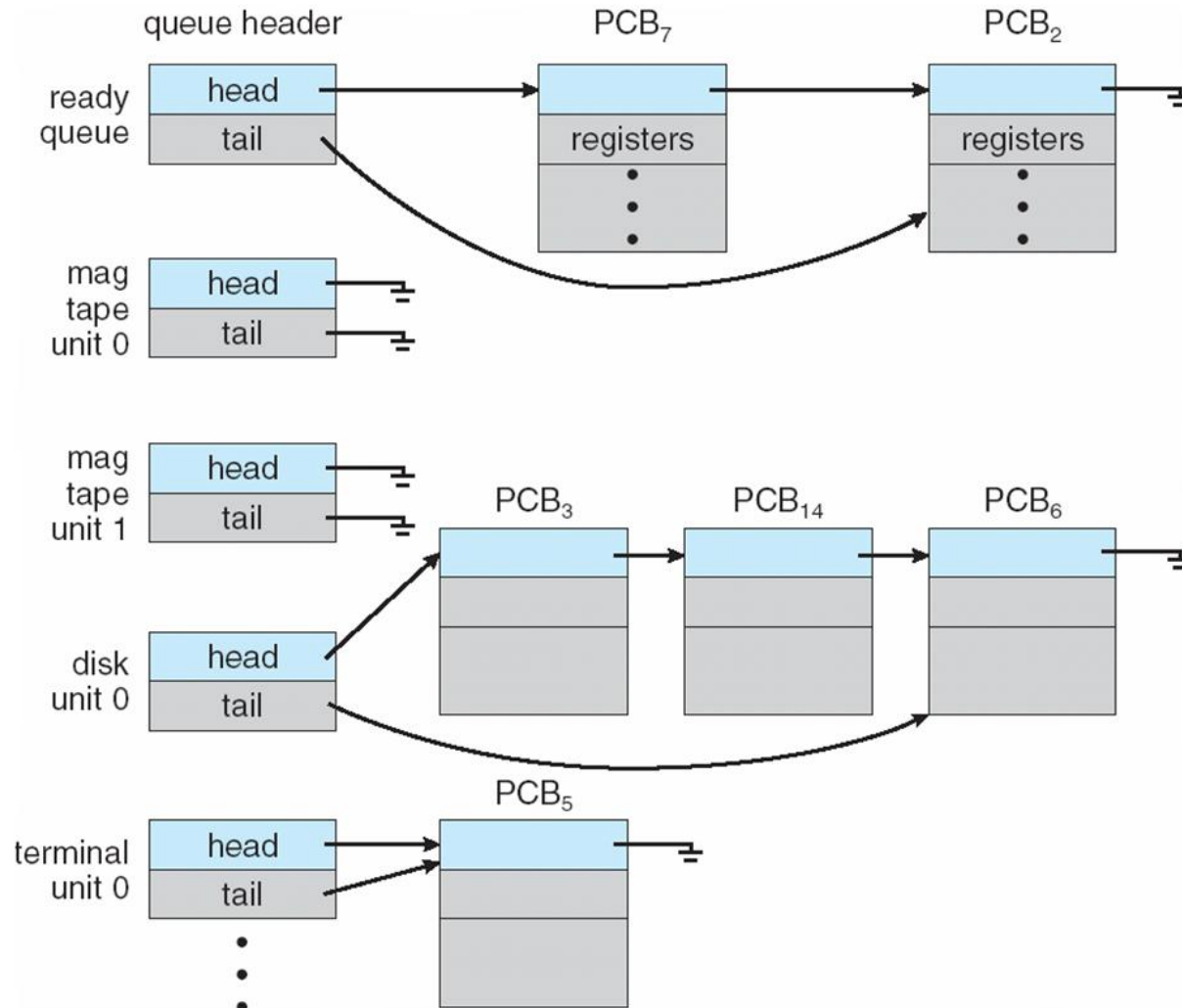
Agendamento de Processos

- ❑ Maximize o uso da CPU, alterne rapidamente os processos para a CPU para compartilhamento de tempo
- ❑ **O agendador de processos** seleciona entre os processos disponíveis para a próxima execução na CPU
- ❑ Mantém **filas de agendamento** de processos
 - ❑ **Fila de trabalhos** – conjunto de todos os processos no sistema
 - ❑ **Fila pronta** – conjunto de todos os processos que residem na memória principal, prontos e aguardando para serem executados
 - ❑ **Filas de dispositivos** – conjunto de processos aguardando um dispositivo de E/S
 - ❑ Os processos migram entre as várias filas





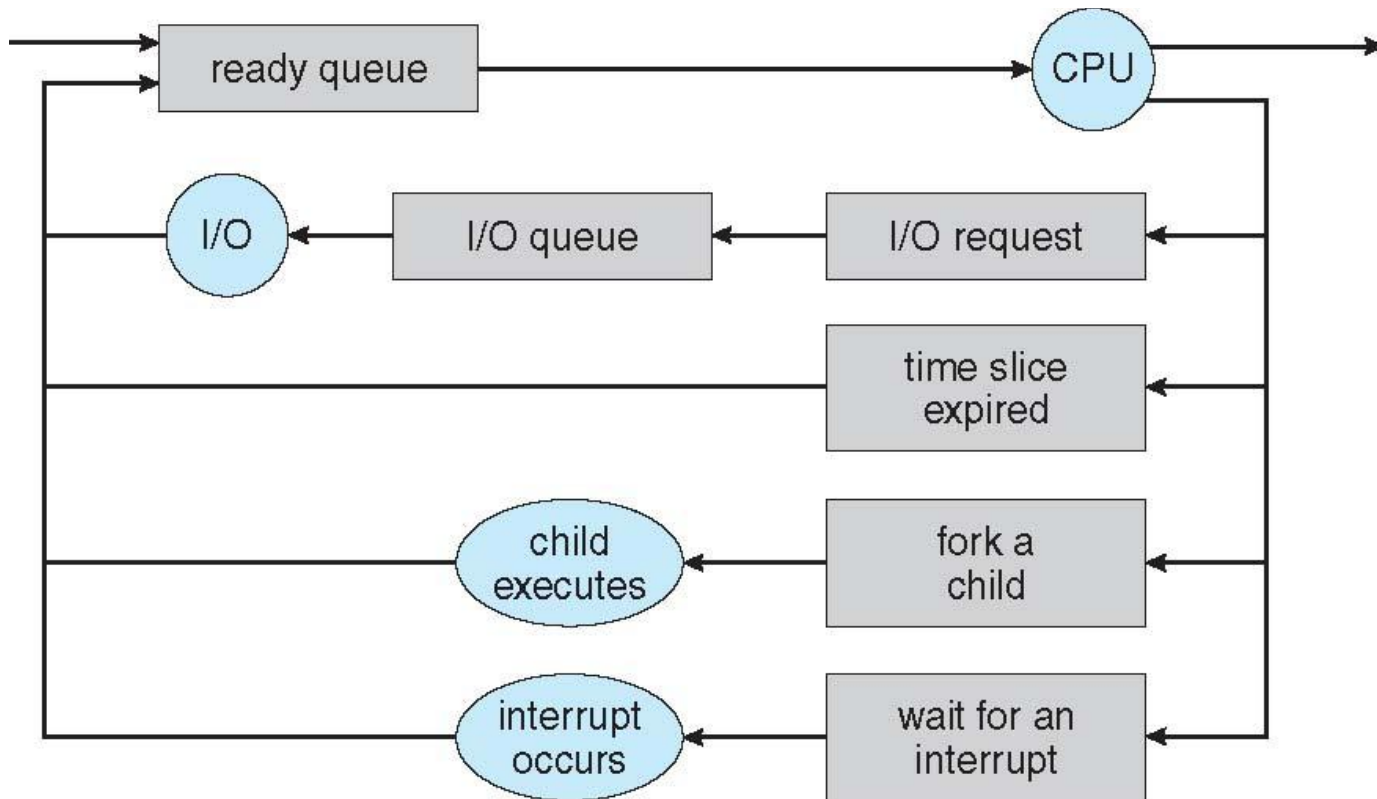
Fila pronta e várias filas de dispositivos de E/S





Representação do Agendamento de Processos

- O diagrama de filas representa filas, recursos e fluxos





Agendadores

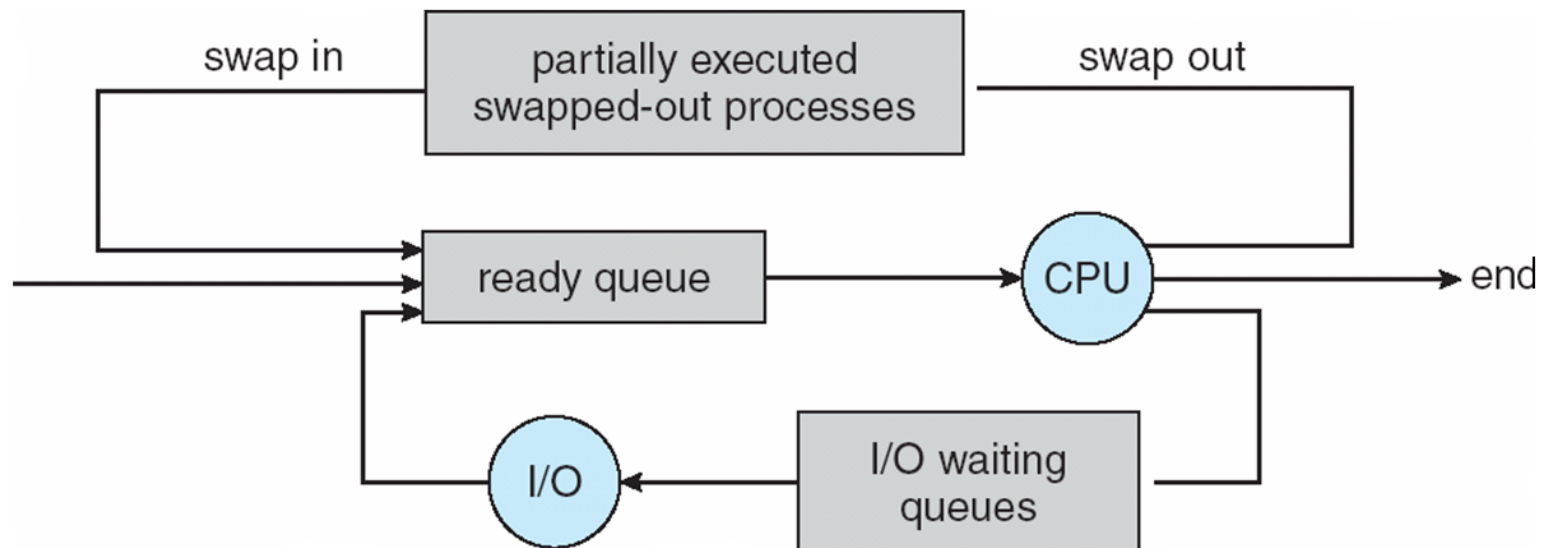
- **Agendador de curto prazo** (ou **agendador de CPU**) – seleciona qual processo deve ser executado em seguida e aloca CPU
 - Às vezes, o único planejador em um sistema
 - O agendador de curto prazo é invocado com frequência (milissegundos) ⇒ (deve ser rápido)
- **Agendador de longo prazo** (ou **agendador de tarefas**) – seleciona quais processos devem ser colocados na fila de espera
 - O agendador de longo prazo é invocado com pouca frequência (segundos, minutos) ⇒ (pode ser lento)
 - O planejador de longo prazo controla o **grau de multiprogramação**
- Os processos podem ser descritos como:
 - **Processo vinculado a E/S** – gasta mais tempo fazendo E/S do que cálculos, muitas explosões curtas de CPU
 - **Processo vinculado à CPU** – gasta mais tempo fazendo cálculos; poucas explosões de CPU muito longas
- O planejador de longo prazo se esforça para obter uma boa **combinação de processos**





Adição de Agendamento de Médio Prazo

- O **agendador de médio prazo** pode ser adicionado se o grau de programação múltipla precisar diminuir
 - Remover processo da memória, armazenar no disco, trazer de volta do disco para continuar a execução:
swapping





Multitarefa em sistemas móveis

- Alguns sistemas móveis (por exemplo, versões iniciais do iOS) permitem que apenas um processo seja executado, outros são suspensos
- Devido ao espaço da tela, os limites da interface do usuário do iOS fornecem um
 - **de primeiro plano** único - controlado pela interface do usuário
 - Vários processos **em segundo plano** – na memória, em execução, mas não na tela e com limites
 - Os limites incluem tarefas únicas e curtas, recebimento de notificações de eventos, tarefas específicas de longa duração, como reprodução de áudio
- O Android roda em primeiro e segundo plano, com menos limites
 - O processo em segundo plano usa um **serviço** para executar tarefas
 - O serviço pode continuar em execução mesmo se o processo em segundo plano for suspenso
 - O serviço não tem interface de usuário e consome pouca memória





Troca de contexto

- ❑ Quando a CPU muda para outro processo, o sistema deve **salvar o estado** do processo antigo e carregar o **estado salvo** para o novo processo por meio de uma **troca de contexto**
- ❑ **Contexto** de um processo representado no PCB
- ❑ O tempo de troca de contexto é uma sobrecarga; o sistema não realiza nenhum trabalho útil durante a troca
 - ❑ Quanto mais complexo o SO e o PCB → mais demorada será a troca de contexto
- ❑ Tempo dependente do suporte de hardware
 - ❑ Alguns hardwares fornecem vários conjuntos de registradores por CPU → vários contextos carregados de uma vez





Operações em Processos

- O sistema deve fornecer mecanismos para:
 - criação de processos,
 - término do processo,
 - e assim por diante, conforme detalhado a seguir





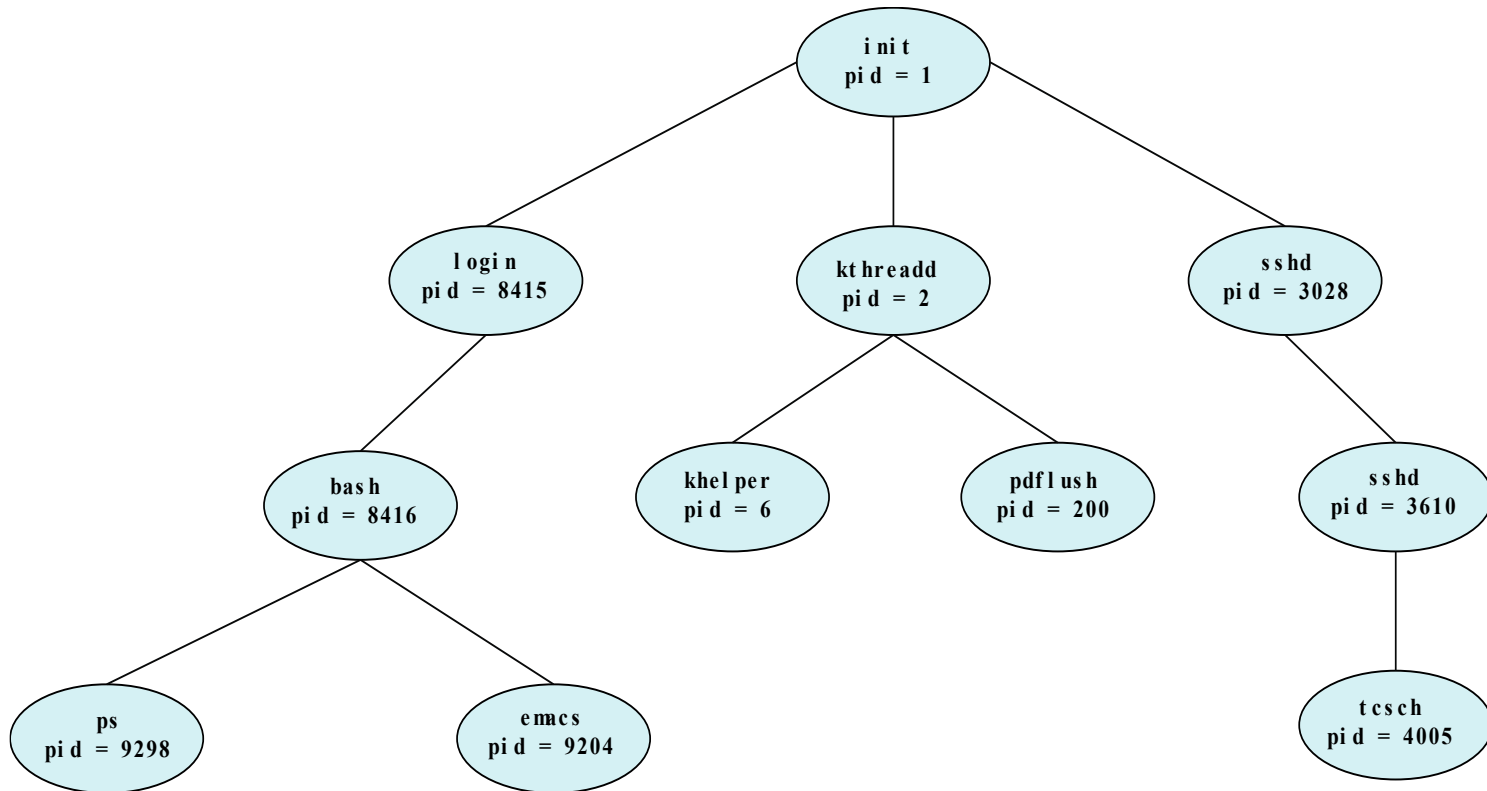
Criação de Processo

- ❑ **Pai** processo criar **filhos** processos, que por sua vez criam outros processos, formando uma **árvore** de processos
- ❑ Geralmente, o processo é identificado e gerenciado por meio de um **identificador de processo** (**pid**)
- ❑ Opções de compartilhamento de recursos
 - ❑ Pais e filhos compartilham todos os recursos
 - ❑ As crianças compartilham um subconjunto dos recursos dos pais
 - ❑ Pais e filhos não compartilham recursos
- ❑ Opções de execução
 - ❑ Pais e filhos executam simultaneamente
 - ❑ Pais esperam até que os filhos terminem





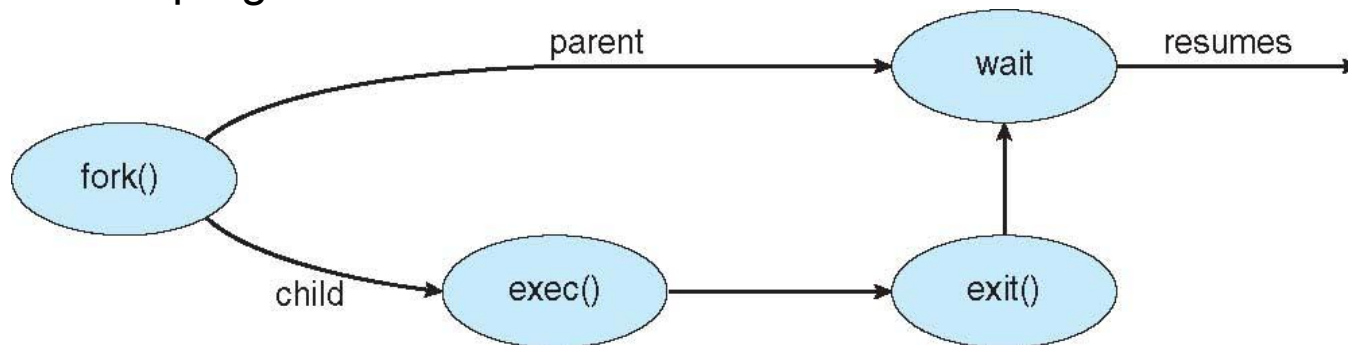
Uma Árvore de Processos no Linux





Criação de Processo (Cont.)

- ❑ Espaço de endereço
 - ❑ Filho duplicado do pai
 - ❑ A criança tem um programa carregado nela
- ❑ Exemplos UNIX
 - ❑ **garfo()** chamada de sistema cria novo processo
 - ❑ chamada de sistema **exec()** usada após um **fork()** para substituir o espaço de memória do processo por um novo programa





Processo separado de bifurcação de programa C

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```





Criando um processo separado via API do Windows

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





Término do Processo

- O processo executa a última instrução e então solicita ao sistema operacional para excluí-la usando a chamada de sistema **exit()** .
 - Retorna dados de status do filho para o pai (via **wait()**)
 - do processo são desalocados pelo sistema operacional
- O pai pode terminar a execução de processos filhos usando a chamada de sistema **abort()** . Alguns motivos para fazer isso:
 - A criança excedeu os recursos alocados
 - A tarefa atribuída à criança não é mais necessária
 - O pai está saindo e o sistema operacional não permite que o filho continue se o pai encerrar





Término do Processo

- Alguns sistemas operacionais não permitem que o filho exista se seu pai tiver terminado. Se um processo termina, todos os seus filhos também devem ser terminados.
 - **término em cascata.** Todos os filhos, netos, etc. são encerrados.
 - O término é iniciado pelo sistema operacional.
- O processo pai pode esperar pelo término de um processo filho usando a chamada de sistema `wait()` . A chamada retorna informações de status e o pid do processo encerrado

`pid = esperar(&status) ;`

- Se nenhum pai estiver esperando (não invocou `wait()`), o processo é um **zumbi**
- Se o pai for encerrado sem invocar `wait` , o processo é **órfão**





Arquitetura Multiprocesso – Navegador Chrome

- ❑ Muitos navegadores da web eram executados como um único processo (alguns ainda o fazem)
 - ❑ Se um site causar problemas, o navegador inteiro pode travar ou travar
- ❑ O navegador Google Chrome é multiprocesso com 3 tipos diferentes de processos:
 - ❑ **do navegador** gerencia a interface do usuário, o disco e a E/S da rede
 - ❑ **de renderização** renderiza páginas da web, lida com HTML, Javascript. Um novo renderizador criado para cada site aberto
 - ▶ Executa em **sandbox** restringindo E/S de disco e rede, minimizando o efeito de explorações de segurança





Comunicação entre processos

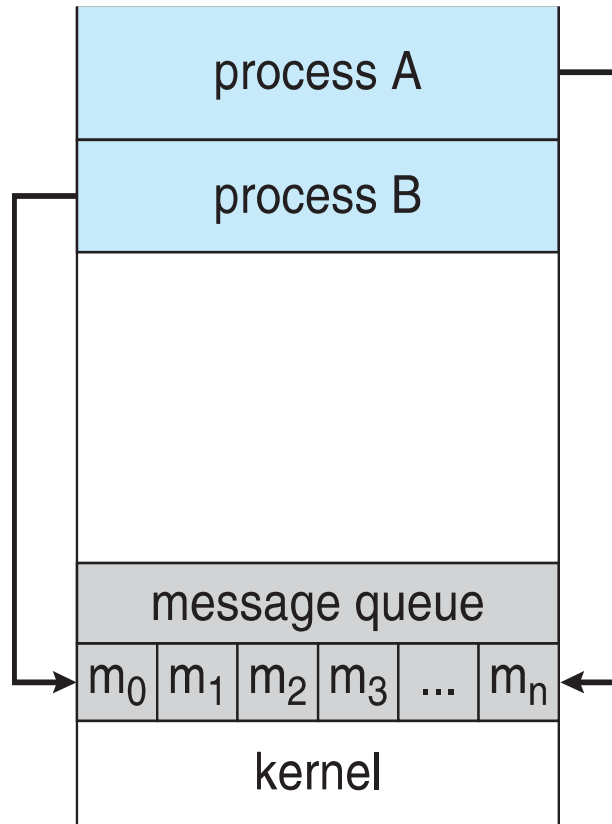
- ❑ Os processos dentro de um sistema podem ser ***independentes*** ou ***cooperando***
- ❑ O processo de cooperação pode afetar ou ser afetado por outros processos, incluindo o compartilhamento de dados
- ❑ Razões para processos cooperativos:
 - ❑ Compartilhamento de informações
 - ❑ Aceleração da computação
 - ❑ Modularidade
 - ❑ Conveniência
- ❑ Os processos cooperativos necessitam de **comunicação entre processos** (**IPC**)
- ❑ Dois modelos de IPC
 - ❑ **Memória compartilhada**
 - ❑ **Passagem de mensagens**



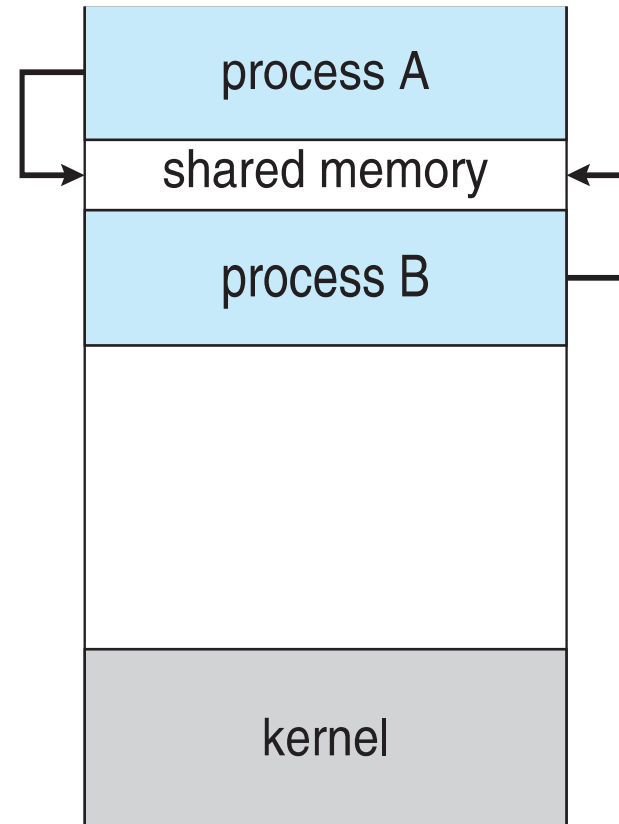


Modelos de Comunicação

(a) Passagem de mensagens. (b) memória compartilhada.

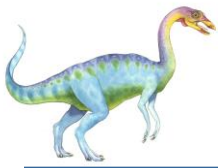


(a)



(b)

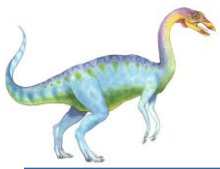




Processos Cooperativos

- **independente** não pode afetar ou ser afetado pela execução de outro processo
- **cooperativo** pode afetar ou ser afetado pela execução de outro processo
- Vantagens da cooperação de processos
 - Compartilhamento de informações
 - Aceleração da computação
 - Modularidade
 - Conveniência





Problema produtor-consumidor

- Paradigma para processos cooperativos, o processo *produtor* produz informações que são consumidas por um processo *consumidor*
 - **unbounded-buffer** não impõe nenhum limite prático ao tamanho do buffer
 - **bounded-buffer** assume que há um tamanho de buffer fixo





Solução de memória compartilhada Bounded-Buffer

□ Dados compartilhados

```
#define TAMANHO_DO_BUFFER 10  
typedef estrutura {  
    . . .  
} item;
```

```
buffer de item[BUFFER_SIZE];  
int em = 0;  
int saída = 0;
```

□ A solução está correta, mas só pode usar elementos BUFFER_SIZE-1

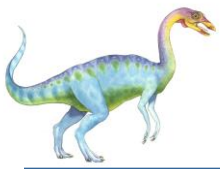




Buffer Limitado – Produtor

```
item next_produced ;  
enquanto (verdadeiro) {  
    /* produzir um item no próximo produzido */  
    enquanto (((entrada + 1) % TAMANHO_DO_BUFFER) == saída)  
        ; /* não faça nada */  
    buffer[in] = próximo_produzido ;  
    em = (em + 1) % TAMANHO_DO_BUFFER;  
}
```





Buffer Limitado – Consumidor

```
item próximo_consumido;  
enquanto (verdadeiro) {  
    enquanto (dentro == fora)  
        ; /* não fazer nada */  
    next_consumed = buffer[out];  
    fora = (fora + 1) % TAMANHO_DO_BUFFER;  
  
    /* consumir o item no próximo consumido */  
}
```





Comunicação entre processos – Memória compartilhada

- ❑ Uma área de memória compartilhada entre os processos que desejam se comunicar
- ❑ A comunicação está sob o controle dos processos dos usuários, não do sistema operacional.
- ❑ A principal questão é fornecer um mecanismo que permita que os processos do usuário sincronizem suas ações quando acessam a memória compartilhada.
- ❑ A sincronização é discutida em detalhes no Capítulo 5.





Comunicação entre processos – Passagem de mensagens

- Mecanismo para que os processos se comuniquem e sincronizem suas ações
- Sistema de mensagens – os processos se comunicam entre si sem recorrer a variáveis compartilhadas
- A instalação do IPC oferece duas operações:
 - **enviar** (*mensagem*)
 - **receber** (*mensagem*)
- O tamanho *da mensagem* é fixo ou variável





Passagem de mensagens (continuação)

- Se os processos P e Q desejam se comunicar, eles precisam:
 - Estabelecer uma **comunicação link** entre eles
 - Trocar mensagens via envio/recebimento
- Problemas de implementação:
 - Como os links são estabelecidos?
 - Um link pode ser associado a mais de dois processos?
 - Quantos links podem existir entre cada par de processos de comunicação?
 - Qual é a capacidade de um link?
 - O tamanho da mensagem que o link pode acomodar é fixo ou variável?
 - Um link é unidirecional ou bidirecional?





Passagem de mensagens (continuação)

- Implementação de link de comunicação
 - Físico:
 - ▶ Memória compartilhada
 - ▶ Barramento de hardware
 - ▶ Rede
 - Lógico:
 - ▶ Direto ou indireto
 - ▶ Síncrono ou assíncrono
 - ▶ Buffering automático ou explícito





Comunicação Direta

- Os processos devem nomear uns aos outros explicitamente:
 - **enviar** (P , *mensagem*) – enviar uma mensagem para processar P
 - **receber** (Q , *mensagem*) – receber uma mensagem do processo Q
- Propriedades do link de comunicação
 - Os links são estabelecidos automaticamente
 - Um link está associado a exatamente um par de processos de comunicação
 - Entre cada par existe exatamente um elo
 - O link pode ser unidirecional, mas geralmente é bidirecional





Comunicação Indireta

- ❑ As mensagens são direcionadas e recebidas de caixas de correio (também chamadas de portas)
 - ❑ Cada caixa de correio tem um ID único
 - ❑ Os processos podem se comunicar somente se compartilharem uma caixa de correio
- ❑ Propriedades do link de comunicação
 - ❑ Link estabelecido somente se os processos compartilharem uma caixa de correio comum
 - ❑ Um link pode estar associado a muitos processos
 - ❑ Cada par de processos pode compartilhar vários links de comunicação
 - ❑ O link pode ser unidirecional ou bidirecional





Comunicação Indireta

□ Operações

- criar uma nova caixa de correio (porta)
- enviar e receber mensagens pela caixa de correio
- destruir uma caixa de correio

□ Primitivos são definidos como:

enviar (*A, mensagem*) – enviar uma mensagem para a caixa de correio A

receber (*A, mensagem*) – receber uma mensagem da caixa de correio A





Comunicação Indireta

- Compartilhamento de caixa de correio
 - P_1 , P_2 e P_3 compartilham a caixa de correio A
 - P_1 , envia; P_2 e P_3 recebem
 - Quem recebe a mensagem?
- Soluções
 - Permitir que um link seja associado a no máximo dois processos
 - Permitir que apenas um processo por vez execute uma operação de recebimento
 - Permitir que o sistema selecione arbitrariamente o destinatário. O remetente é notificado sobre quem foi o destinatário.





Sincronização

- ❑ A passagem de mensagens pode ser bloqueadora ou não bloqueadora
- ❑ **O bloqueio** é considerado **síncrono**
 - ❑ **Bloqueando envio** -- o remetente é bloqueado até que a mensagem seja recebida
 - ❑ **Bloqueando recebimento** -- o receptor é bloqueado até que uma mensagem esteja disponível
- ❑ **Não bloqueante** é considerado **assíncrono**
 - ❑ **Envio sem bloqueio** - o remetente envia a mensagem e continua
 - ❑ **Recepção não bloqueante** — o receptor recebe:
 - ❑ Uma mensagem válida, ou
 - ❑ Mensagem nula
- ❑ Diferentes combinações possíveis
 - ❑ Se tanto o envio quanto o recebimento estiverem bloqueados, temos um **encontro**





Sincronização (Cont.)

□ Produtor-consumidor torna-se trivial

```
mensagem next_produced ;  
enquanto (verdadeiro) {  
    /* produz um item no próximo produzido */  
    enviar( próximo_produzido );  
}
```

```
mensagem next_consumed;  
enquanto (verdadeiro) {  
    receber(próximo_consumido);  
  
    /* consumir o item no próximo consumido */  
}
```





Buffering

- Fila de mensagens anexadas ao link.
- implementado de uma das três maneiras
 1. Capacidade zero – nenhuma mensagem é enfileirada em um link.
O remetente deve esperar pelo destinatário (rendezvous)
 2. Capacidade limitada – comprimento finito de n mensagens
O remetente deve esperar se o link estiver cheio
 3. Capacidade ilimitada – comprimento infinito
O remetente nunca espera





Exemplos de sistemas IPC - POSIX

? Memória compartilhada POSIX

? O processo primeiro cria o segmento de memória compartilhada
`shm_fd = shm_open (nome, O_CREAT | O_RDWR,
0666) ;`

? Também usado para abrir um segmento existente para compartilhá-lo

? Defina o tamanho do objeto

```
ftruncate ( shm fd , 4096) ;
```

? Agora o processo pode gravar na memória compartilhada
`sprintf (memória compartilhada, "Escrevendo na
memória compartilhada") ;`





Produtor IPC POSIX

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

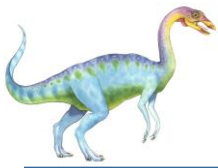
    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```





Consumidor IPC POSIX

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```





Exemplos de sistemas IPC - Mach

- A comunicação Mach é baseada em mensagens
 - Até mesmo chamadas de sistema são mensagens
 - Cada tarefa recebe duas caixas de correio na criação - Kernel e Notify
 - Apenas três chamadas de sistema são necessárias para transferência de mensagens

`msg_send()` , `msg_receive()` , `msg_rpc()`

- Caixas de correio necessárias para comunicação, criadas via `porta_alocar()`
- Enviar e receber são flexíveis, por exemplo, quatro opções se a caixa de correio estiver cheia:
 - ▶ Espere indefinidamente
 - ▶ Aguarde no máximo n milissegundos
 - ▶ Retornar imediatamente
 - ▶ Armazenar uma mensagem em cache temporariamente





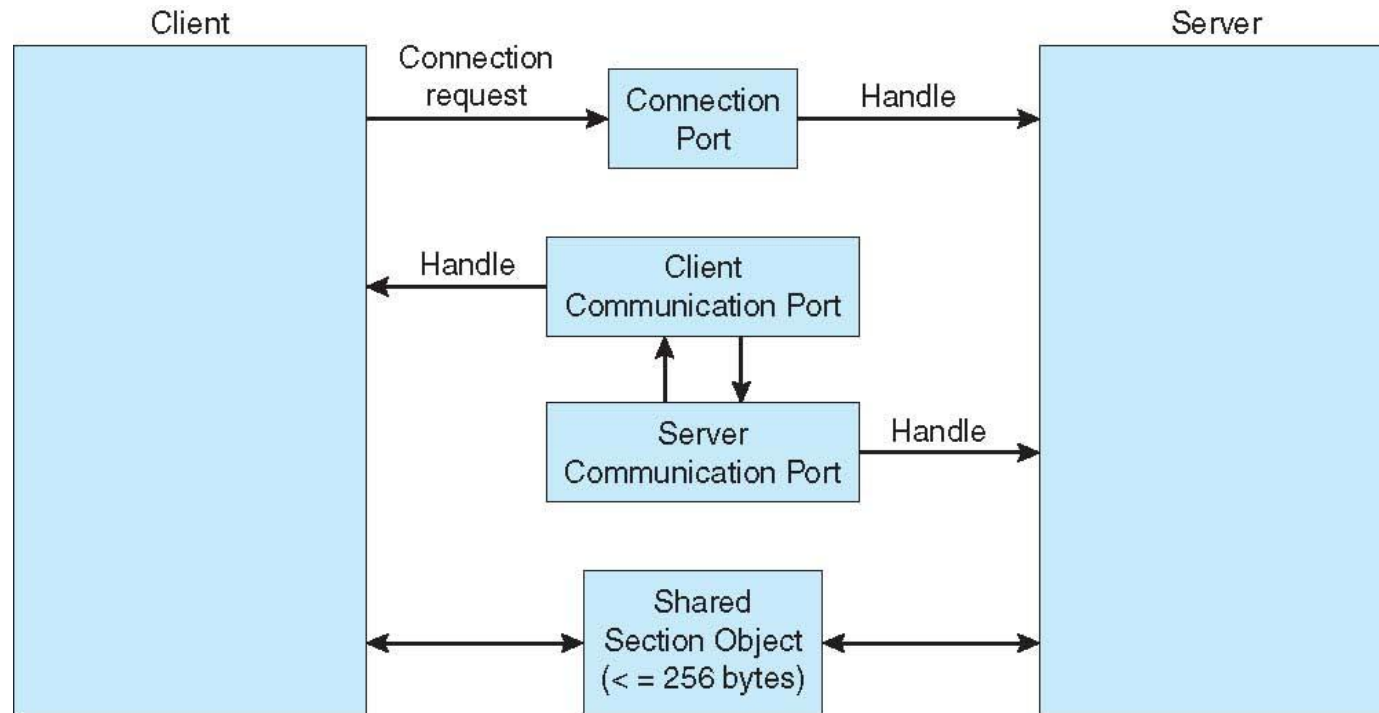
Exemplos de sistemas IPC – Windows

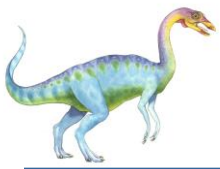
- Centralização de passagem de mensagens por meio de **recurso avançado de chamada de procedimento local (LPC)**
 - Funciona apenas entre processos no mesmo sistema
 - Usa portas (como caixas de correio) para estabelecer e manter canais de comunicação
 - A comunicação funciona da seguinte forma:
 - ▶ O cliente abre um identificador para o objeto **de porta de conexão** do subsistema .
 - ▶ O cliente envia uma solicitação de conexão.
 - ▶ O servidor cria duas **portas de comunicação privadas** e retorna o identificador de uma delas ao cliente.
 - ▶ O cliente e o servidor usam o identificador de porta correspondente para enviar mensagens ou retornos de chamada e para ouvir respostas.





Chamadas de Procedimento Local no Windows





Comunicações em sistemas cliente-servidor

- ❑ Soquetes
- ❑ Chamadas de Procedimento Remoto
- ❑ Tubos
- ❑ Invocação de método remoto (Java)





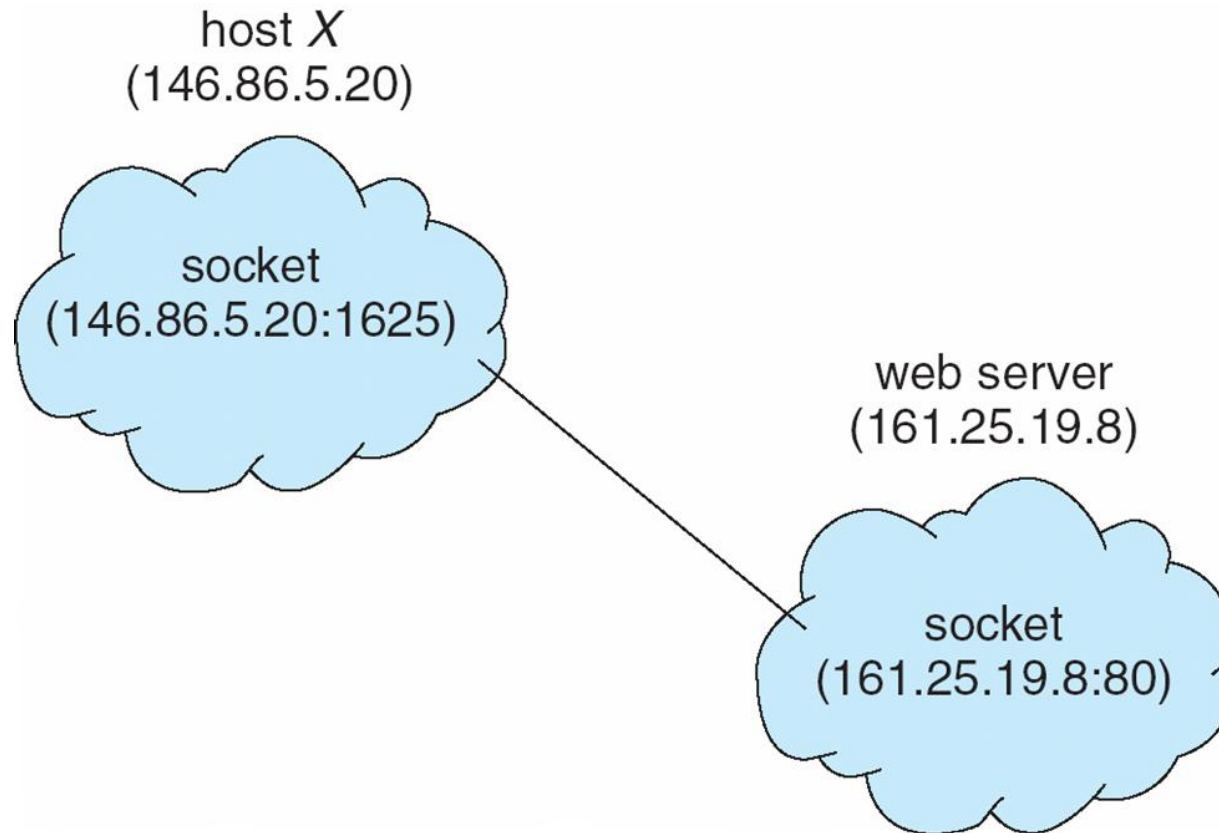
Soquetes

- Um **soquete** é definido como um ponto final para comunicação
- Concatenação de endereço IP e **porta** – um número incluído no início do pacote de mensagens para diferenciar serviços de rede em um host
- O soquete **161.25.19.8:1625** refere-se à porta **1625** no host **161.25.19.8**
- A comunicação consiste entre um par de soquetes
- Todas as portas abaixo de 1024 são ***bem conhecidas*** e usadas para serviços padrão
- Endereço IP especial 127.0.0.1 (**loopback**) para se referir ao sistema no qual o processo está sendo executado





Comunicação de soquete





Soquetes em Java

- Três tipos de soquetes
 - **Orientado a conexão (TCP)**
 - **Sem conexão (UDP)**
 - **MulticastSocket** – os dados podem ser enviados para vários destinatários
- Considere este servidor “Data”:

```
import java.net.*;
import java.io.*;

public class DateServer
{
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);

            /* now listen for connections */
            while (true) {
                Socket client = sock.accept();

                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(), true);

                /* write the Date to the socket */
                pout.println(new java.util.Date().toString());

                /* close the socket and resume */
                /* listening for connections */
                client.close();
            }
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```





Chamadas de Procedimento Remoto

- Chamada de procedimento remoto (RPC) abstrai chamadas de procedimento entre processos em sistemas em rede
 - Novamente usa portas para diferenciação de serviço
- **Stubs** – proxy do lado do cliente para o procedimento real no servidor
- O stub do lado do cliente localiza o servidor e **organiza** os parâmetros
- O stub do lado do servidor recebe esta mensagem, descompacta os parâmetros marshalled e executa o procedimento no servidor
- No Windows, o código stub é compilado a partir de especificações escritas em **Microsoft Interface Definition Language** (**MIDL**)





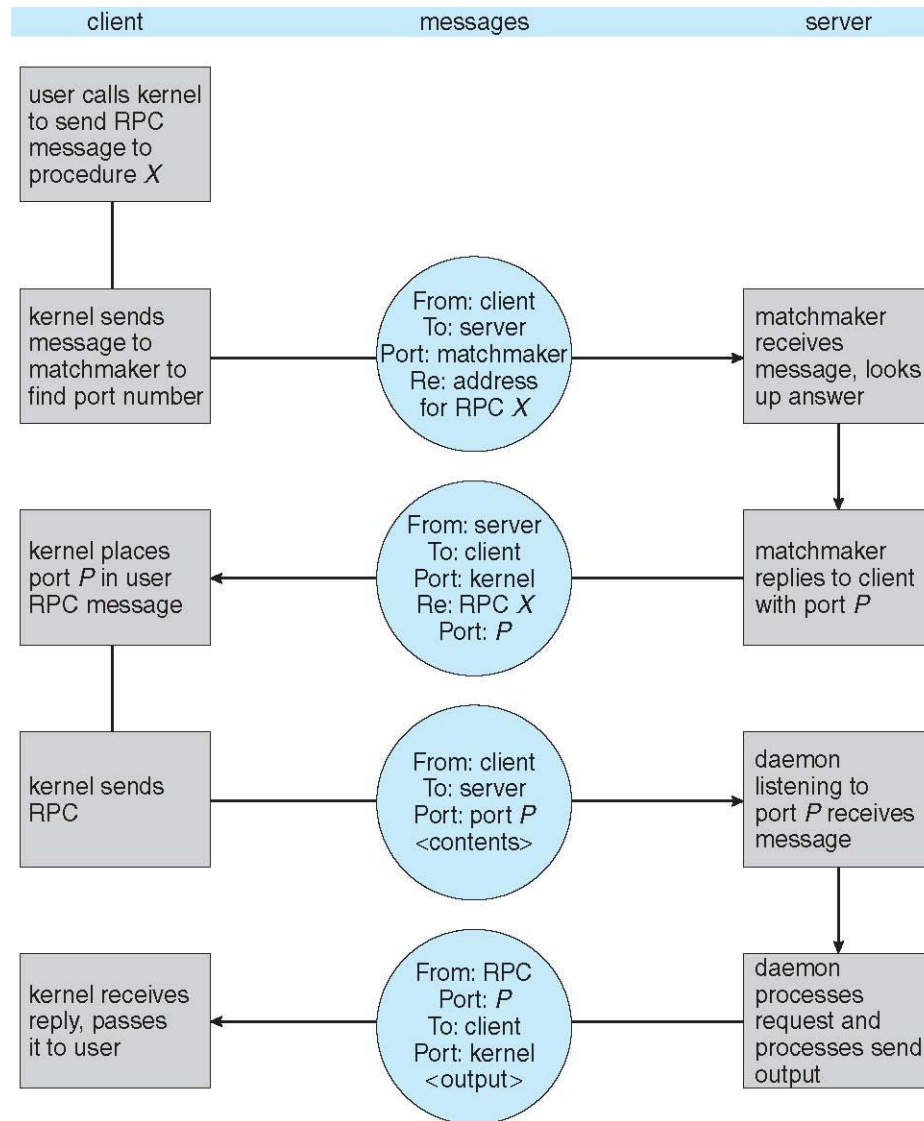
Chamadas de Procedimento Remoto (Cont.)

- Representação de dados manipulada via formato de **Representação de Dados Externos (XDL)** para dar conta de diferentes arquiteturas
 - **Big-endian** e **little-endian**
- A comunicação remota tem mais cenários de falha do que a comunicação local
 - As mensagens podem ser entregues ***exatamente uma vez***, em vez de ***no máximo uma vez***
- O sistema operacional normalmente fornece um serviço de encontro (ou **matchmaker**) para conectar cliente e servidor





Execução do RPC





PIPES

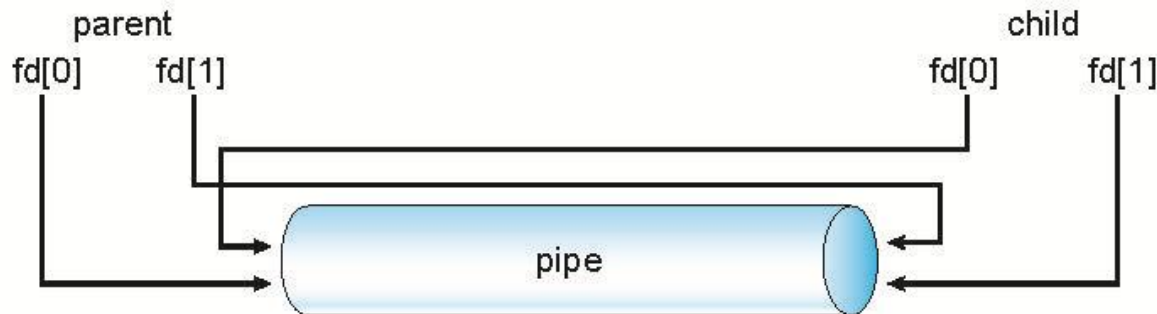
- Atua como um canal permitindo que dois processos se comuniquem
- Problemas:
 - A comunicação é unidirecional ou bidirecional?
 - No caso de comunicação bidirecional, é half ou full-duplex?
 - Deve existir uma relação (ou seja, ***pai-filho***) entre os processos de comunicação?
 - Os tubos podem ser usados em rede?
- Pipes comuns – não podem ser acessados de fora do processo que os criou. Normalmente, um processo pai cria um pipe e o usa para se comunicar com um processo filho que ele criou.
- Pipes nomeados – podem ser acessados sem um relacionamento pai-filho.





Tubos comuns

- ? Tubos comuns permitir a comunicação no estilo padrão produtor-consumidor
- ? O produtor escreve em uma extremidade (a **extremidade de gravação** do pipe)
- ? O consumidor lê do outro lado (o **lado da leitura**) do tubo)
- ? Os tubos comuns são, portanto, unidirecionais
- ? Exige relação pai-filho entre processos de comunicação



- ? O Windows chama esses **pipes anônimos**
- ? Veja exemplos de código Unix e Windows no livro didático





Pipes nomeados

- ❑ Pipes nomeados são mais poderosos que pipes comuns
- ❑ A comunicação é bidirecional
- ❑ Não é necessária nenhuma relação pai-filho entre os processos de comunicação
- ❑ Vários processos podem usar o pipe nomeado para comunicação
- ❑ Fornecido em sistemas UNIX e Windows



Fim do Capítulo 3

